

곤지암리조트 식음업장 수요예측 — 팀 브리핑 2편

DACON 236559 · LG AIMERS 7기 | 8월 2~3일 작업 결과

이전 성적	Public 0.4818 · 55등 (1편 시점)
현재 성적	Public 0.4594 · Private 0.4439 · 대회 3등
전제	1편(2026-08-01)을 읽었다고 가정합니다. 창 · origin · horizon · SMAPE는 1편 1장에 있습니다.
출처	experiments/log.md · 실험 기록 12건 · 제출 9건의 Public/Private 점수

- 성적이 0.4818 → 0.4516으로 올랐는데, 개선의 거의 전부가 "모델을 좋게 만든 것"이 아니라 "채점 방식을 이용한 것"에서 나왔습니다. 모델은 사실상 그대로입니다.
- 우리가 쓰던 내부 검증 점수가 실제 대회 성적과 반대로 움직이고 있었습니다. 내부 검증이 1등으로 뽑은 설정이 실제로는 5등이었습니다. 원인을 찾아 검증 방식을 바꿨습니다.
- 모델을 바꾸는 시도 세 가지는 모두 성적이 나아지지 않았습니다. 설정을 79가지 시험해서 얻은 순이익이 사실상 0입니다. 무엇을 왜 시도했고 왜 안 됐는지 5장에 남깁니다.

0. 이 문서에 나오는 말 정리

1편에서 정의한 것(창 · origin · horizon · SMAPE · 피쳐)은 다시 설명하지 않습니다. 이번 문서에서 새로 나오거나 자주 쓰이는 말만 정리합니다.

말	뜻	일반 용어인가
폴드 (fold)	검증 한 회차. 가진 데이터의 일부를 일부러 감춰두고, 나머지로 학습시킨 뒤 감춘 곳에서 맞춰보게 하는 것. 이걸 감추는 위치를 바꿔가며 여러 번 하는데, 그 한 번이 폴드 하나입니다.	머신러닝 표준 용어
F2 · F3 FAR-봄 · FAR-겨울	우리가 쓰는 폴드 4개의 이름. F2는 겨울을 감춰두고 검증하는 회차, F3는 봄. FAR로 시작하는 둘은 학습 시점과 검증 시점 사이를 3개월 벌려둔 회차입니다 (6장에서 설명).	우리가 붙인 이름
교차검증	일부 폴드에서 값을 정하고, 나머지 폴드에서 채점하는 것. 같은 데이터에서 값을 정하고 그 데이터로 채점하면 항상 좋게 나오므로, 정한 곳과 채점하는 곳을 분리합니다.	머신러닝 표준 용어
자유도	우리가 조절할 수 있는 숫자의 개수. 예: 예측값 전체에 숫자 하나를 곱하면 자유도 1, 구간을 셋으로 나눠 각각 다른 숫자를 곱하면 자유도 3입니다. 자유도가 클수록 검증 데이터에는 잘 맞지만 실제 성적은 나빠지기 쉽습니다.	통계 표준 용어
분해능	구분할 수 있는 최소 차이. 우리 측정으로는 0.0032보다 작은 차이는 "누가 더 나은지 판단 불가"입니다. 저울에 눈금이 1g 단위면 0.5g 차이는 못 읽는 것과 같습니다.	우리가 쓰는 표현 (측정 분야에서 빌려옴)
시드	학습할 때 굴리는 주사위의 번호. LightGBM은 트리마다 데이터와 피쳐의 일부만 무작위로 골라 쓰는데, 시드가 그 무작위를 정합니다. 시드가 다르면 같은 데이터·같은 설정에서도 조금 다른 모델이 나옵니다.	머신러닝 표준 용어
내부 검증 점수	폴드에서 낸 점수. 대회에 제출하지 않고 우리끼리 매기는 점수입니다. 이 문서의 핵심 사건이 "이 점수가 실제 대회 성적과 반대로 움직였다"입니다.	—
후보정	모델이 낸 예측값을 나중에 손보는 것. 모델 자체는 그대로 두고 출력 숫자만 바꿉니다.	—
압축	모델이 작은 값은 실제보다 크게, 큰 값은 실제보다 작게 예측하는 현상. 예측값들이 가운데로 몰린다는 뜻입니다. 4장의 핵심입니다.	—

1. 성적이 어떻게 움직였나

1.1 먼저 Public과 Private

대회는 테스트 정답을 알려주지 않고, 채점 결과도 절반만 보여줍니다. 보여주는 쪽이 **Public**, 숨겨두는 쪽이 **Private**이며 **최종 등수는 Private으로** 매깁니다. 제출을 반복해서 리더보드에 맞추는 것을 막으려는 장치입니다. 이번에 Private 점수가 전체 제출건에 대해 공개됐습니다. 두 반쪽을 합치면 **테스트 전체 성적**이 나오고, 그게 우리가 실제로 비교해야 할 숫자입니다.

제출	무엇을 바꿨나	Public	Private	합산(실제)
v1	1편 시점 기준 모델 (LightGBM, 트리 잎 255개)	0.4818	0.4607	0.47126
v3	XGBoost로 교체하고 설정 40가지 탐색	0.4804	0.4619	0.47119
v4	LightGBM 재설정 — 트리를 크게 (잎 511개)	0.4871	0.4658	0.47644
v5	v3과 v4의 예측을 평균	0.4833	0.4631	0.47321
v6	트리를 작게 (잎 15개)	0.4818	0.4584	0.47008
v7	잎 255개 + 분기마다 피처의 65%만 사용	0.4827	0.4612	0.47195
v8	잎 127개 + 분기마다 피처의 65%만 사용	0.4807	0.4570	0.46883
v9	v8과 똑같은 모델 + 예측값 후보정	0.4594	0.4439	0.45161
v10	후보정 중 낮은 구간만 적용	0.4636	0.4473	0.45544

이 표의 요점

v1부터 v8까지 모델을 일곱 번 바꿨는데 합산 점수는 **0.47126 → 0.46883, 0.0024밖에 안 움직였습니다**. 그런데 **v8 → v9는 모델을 전혀 안 건드리고 예측값에 숫자만 곱했는데 0.0172 좋아졌습니다. 7배 차이**입니다.

1.2 얼마나 차이나야 진짜 차이인가 — 0.0032

Public과 Private은 같은 모델을 서로 다른 절반으로 채점한 것입니다. 두 점수의 차이를 제출 9건 전부에서 재보니 평균 +0.021, 흩어진 정도(표준편차)가 0.0016이었습니다. 그 두 배인 **0.0032**를 기준선으로 씁니다.

즉 두 모델의 점수 차이가 **0.0032보다 작으면, 어느 쪽이 나은지 우리는 판단할 수 없습니다**. 채점 대상이 바뀌면 순위가 뒤집힐 수 있는 크기라는 뜻입니다.

모델 묶음	점수 폭	판정
v8 · v6 · v3 · v1 (상위 4개)	0.0019	구분 불가 — 사실상 동률
v5 (두 모델 평균)	v8 대비 0.0044	경계선
v4 (트리 키운 것)	v8 대비 0.0076	확실히 나쁨
v9 (후보정)	v8 대비 0.0172	기준선의 5배 — 확실

이 숫자가 중요한 이유: 상위 4개 안에 **1편 시점의 기준 모델 v1**이 들어 있습니다. 모델을 아무리 바꿔도 출발점에서 못 벗어났다는 뜻이고, 앞으로 무엇을 하든 **0.0032**를 못 넘으면 판정 자체가 불가능하다는 뜻입니다.

2. 모델이 실제로 무엇을 보는가

5장(모델 구조 시도)을 읽으려면 이게 먼저 필요합니다. 1편에는 피쳐 목록이 있지만 "한 번 예측할 때 무엇을 보는가"는 정리돼 있지 않았습니다.

2.1 학습 데이터 한 줄은 무엇인가

학습 데이터 한 줄 = (품목 하나, origin 하나, horizon 하나)

예) 품목 = 담하_불고기
origin = 2024-03-01 (창은 2/3 ~ 3/1)
horizon = 3

입력: 위 상황을 요약한 숫자 57개

정답: 2024-03-04 의 담하_불고기 판매량

이런 줄이 **32만 개** 있습니다(품목 193 × origin 498 × horizon 7 중 정답이 0이 아닌 것).

2.2 그 57개는 어디서 오는가

출처	개수	구체적으로
그 품목 자신	약 30	28일 창의 평균 · 최댓값 · 표준편차 · 최근 7일 평균 · 같은 요일 평균 · 지난주 같은 요일 값 · 마지막 판매 이후 경과일 등
같은 영업장 전체	약 14	담하 일별 합계 · 담하 최근 7일 대비 그 이전 7일 · 담하 휴점 비율 · 불고기가 담하 매출에서 차지하는 비율 · 담하의 '인원수 대표 품목'(공깃밥) 판매량
달력	13	요일 · 월 · 공휴일 · 화담숲 개장기 · 스키 시즌 등. 품목과 무관한 날짜 정보
품목 고정 속성	7	영업장 · 군집 · 분류 · 가격 등급 · item_id
다른 영업장	0	없습니다

담하 불고기를 예측할 때, 리조트의 다른 가게 정보는 전혀 안 봅니다

미라시아가 오늘 붐볐는지, 스키장에 사람이 많은지는 **입력에 아예 없습니다**. 담하 안의 정보만 봅니다.

이건 개선 여지입니다. "오늘 리조트 전체가 붐빈다"는 정보를 지금 모델은 **구조적으로 알 수가 없습니다**. 자기 영업장 합계가 그걸 간접적으로 담기는 하지만, "리조트가 붐빔"과 "우리 가게만 붐빔"이 섞여 있습니다. 미시도 항목으로 9장에 올려놨습니다.

2.3 그래서 "글로벌 모델"과 "전용 모델"의 차이는

입력 피쳐는 완전히 같습니다. 달라지는 건 학습에 쓰는 줄뿐입니다.

글로벌 모델 : 193품목 전부의 줄 32만 개로 모델 1개를 만든다
(담하 줄도, 연회장 줄도, 화당숲 줄도 한 덩어리)

연회장 전용 : 연회장 23품목의 줄 3.8만 개로만 모델 1개를 만든다
(담하 줄은 아예 안 본다)

전용 모델에 기대했던 이점은 "트리의 분기를 연회장 패턴에만 쓸 수 있다"였습니다. 결과와 이유는 5.3에 있습니다.

3. 오늘 한 작업 요약

작업	규모	실제 효과	결과
예측값 후보정 (구간별로 다른 숫자 곱하기)	계산 5분	+0.0172	채택
모델 설정 탐색	설정 79가지 · 약 5시간	약 0	성적 변화 없음
서로 다른 알고리즘의 예측을 평균	측정 3회	약 0	성적 변화 없음
모델을 영업장별 · 날짜별로 쪼개기	26분	약 0	성적 변화 없음
학습할 때 줄마다 다른 비중 주기	5단계 · 44분	약 0	성적 변화 없음
내부 검증 방식 재설계	1시간	간접	핵심 성과

한 가지 규칙성이 보입니다

1편에서 확정된 개선 6개 중 큰 것 셋 — 정답이 0인 줄 제거(0.352), $\log_{1p} + L1$, 예측 하한 1.0 — 은 전부 "채점이 실제로 무엇을 재는가"를 이용한 것이었습니다. 오늘 성적을 올린 후보정(+0.0172)도 같은 종류입니다.

반대로 모델 자체를 좋게 만들려는 시도는 1편의 ⑤(트리 크게, +0.004) 이후로 오늘 네 갈래 전부 성적이 나아지지 않았습니다.

이 문제에서는 "모델을 개선하는 것"보다 "채점 방식을 이용하는 것"이 훨씬 크게 작용합니다.

4. 성적이 크게 오른 것 — 예측값 후보정 (+0.0172)

4.1 무엇을 했나

모델이 낸 예측값에 크기별로 다른 숫자를 곱했습니다. 모델은 전혀 안 건드렸습니다.

예측값이 2 미만이면 → × 0.55
예측값이 2 ~ 10 이면 → × 0.90
예측값이 10 이상이면 → × 1.02
그 다음 하한 1.0 적용

이게 전부입니다. 학습이 없고 저장된 예측 파일을 읽어 곱하기만 하면 되니 몇 초 걸립니다.

4.2 어떻게 찾았나

원래 질문은 다른 것이었습니다. "하한을 1이 아니라 1과 2 사이로 하면 더 좋지 않을까?" 채점 대상의 34%가 실측 1(21%) 또는 2(13%)이니, 1로 눌러두면 실측이 2인 칸에서 손해가 크다는 생각이었습니다.

재본 결과 하한은 1.00이 정확히 최적이었고, 폴드 4개가 모두 같은 답을 냈습니다.

하한	0.90	0.95	1.00	1.05	1.20	1.50	2.00
점수	0.5082	0.5082	0.5079	0.5093	0.5161	0.5304	0.5482

왜 예상이 빗나갔나: 하한이 적용되는 것은 전체 칸이 아니라 모델이 낮게 예측한 칸뿐입니다. "실측 2가 13%"는 전체 기준이고, 모델이 낮게 본 칸들 안에서는 실측이 대부분 정말 1이었습니다. 모델의 예측 자체가 이미 정보를 담고 있었던 것입니다.

그 옆에서 떨어져 나온 진단이 진짜였습니다. 예측값을 크기순으로 10등분해서 각 구간마다 "곱했을 때 가장 좋은 숫자"를 따로 재본 것입니다.

예측값 구간	그 구간의 실측 중앙값	그 구간의 예측 중앙값	가장 좋은 배율	뜻
0.63 ~ 1.44	1.0	1.19	0.60	실제보다 크게 예측 중
1.44 ~ 1.88	2.0	1.67	0.60	
2.38 ~ 3.04	3.0	2.69	0.82	
5.72 ~ 9.49	7.0	7.16	1.00	거의 정확
19.6 ~ 46.2	36.0	30.05	1.17	실제보다 작게 예측 중
46.2 ~ 815	85.0	73.11	1.10	

모델 예측이 가운데로 몰려 있었습니다 (= 압축)

작은 값은 실제보다 크게, 큰 값은 실제보다 작게 예측하고 있었습니다. 회귀 모델의 일반적인 성질입니다 — 확신이 없으면 평균 쪽으로 붙습니다.

이 발견 전에 "예측값 전체에 같은 숫자 하나를 곱하기"도 시도했는데 효과가 없었습니다. 이유가 이것입니다 — 구간마다 필요한 방향이 반대라 숫자 하나로 잡을 수가 없습니다.

4.3 왜 이런 일이 생기나

두 가지가 겹칩니다.

첫째, 우리 모델은 "중앙값"을 냅니다. `log1p` + L1으로 학습하면 그렇게 됩니다(1편 ②). 그런데 SMAPE가 원하는 값은 중앙값보다 아래입니다. 1편에 이미 증거가 있었습니다 — 모든 칸을 같은 숫자로 찍을 때 최적이 3인데, 실측의 중앙값은 4였습니다.

왜 아래인가: SMAPE 벌점은 아무리 크게 틀려도 최대 2.0에서 멈춥니다. 예측을 4에서 3으로 내리면:

실측	예측 4일 때 벌점	예측 3일 때 벌점	변화
50 (큰 값)	1.704	1.774	0.070 손해
2 (작은 값)	0.667	0.400	0.267 이득

큰 값에서 잃는 것보다 작은 값에서 얻는 것이 4배 큼니다. 실측 50짜리는 3으로 찍든 4로 찍든 이미 벌점 상한 근처라 어차피 크게 틀린 것이고, 실측 2짜리는 3과 4가 진짜로 차이납니다. SMAPE는 "어차피 못 맞히는 큰 값은 포기하고, 맞힐 수 있는 작은 값에 맞춰라"를 유도합니다.

둘째, 압축 자체. 위 표에서 본 대로 큰 값은 작게 예측돼 있으니 살짝 올려야 합니다. 첫째와 둘째가 방향이 반대라, 구간을 나눠야만 둘 다 잡힙니다.

4.4 효과의 78%가 한 구간에서 나옵니다

어디에 적용	내부 검증 개선	폴드 4개 모두 개선	값이 바뀌는 칸
세 구간 전부 (0.55, 0.90, 1.02)	+0.0086	○	97.1%
낮은 구간만 (0.55, 1.00, 1.00)	+0.0067	○	35.7%
중간 구간만 (1.00, 0.90, 1.00)	+0.0015	×	39.7%
높은 구간만 (1.00, 1.00, 1.02)	+0.0004	×	21.8%

예측값 2 미만을 0.55배 하는 것 하나가 효과의 78%입니다. 하한 1.0과 맞물려서 예측값 1.82 미만이 전부 정확히 1.0이 됩니다(전체 칸의 34%). 실체는 이 한 문장입니다:

"낮게 예측한 칸은 그냥 1로 찍어라."

실제 대회 점수로도 확인됐습니다 — 세 구간 전부 적용한 v9(0.45161)가 낮은 구간만 적용한 v10(0.45544)보다 0.0038 좋습니다. 0.0032를 넘으니 중간·높은 구간 보정도 값어치가 있긴 합니다.

4.5 이번엔 내부 검증이 절반으로 과소평가했습니다

	내부 검증 예상	실제	배율
v9 (세 구간 전부)	+0.0086	+0.0172	2.0배
v10 (낮은 구간만)	+0.0067	+0.0134	2.0배
v9 - v10 (중간·높은 구간의 기여)	+0.0019	+0.0038	2.0배

세 값 모두 정확히 2배입니다. 형태는 맞았고 크기만 일정하게 커졌습니다.

추정 원인: 우리 폴드는 3개월 구간이라, 그 기간에 안 팔린 계절 품목은 채점에서 통째로 빠집니다. 실제 대회 채점은 1년 전체라 **193개 품목이 전부 유효 날짜를 갖고 각자 한 표씩** 받습니다. 그렇게 추가로 들어온 품목들이 대부분 판매량이 적은 = 예측값이 낮은 칸이고, **후보정이 가장 크게 작용하는 영역**입니다.

확인 결과 — 채점 방식을 고치니 보정값이 실제로 달라졌습니다

폴드를 각각 채점하는 대신 **겨울과 봄을 이어붙여 한 번에 채점**하면 실제 대회 구조에 가까워집니다(겨울 폴드 단독으로는 품목 5개가 채점에서 빠지는데, 이어붙이면 193개가 모두 채점됩니다). 그 방식으로 보정값을 다시 맞춰봤습니다.

채점 방식	다시 맞춘 배율	개선
폴드를 각각 채점 (기준)	(0.57, 0.90, 1.05)	+0.0089
이어붙여 채점 (교정)	(0.60, 0.93, 1.18)	+0.0118

높은 구간의 배율이 1.05에서 1.18로 크게 달라졌습니다. 즉 지금 쓰고 있는 v9의 1.02는 왜곡된 채점 방식에서 나온 값이었고, 큰 예측값을 올려야 하는 정도를 상당히 과소평가하고 있었습니다.

교차검증으로도 확인했습니다 — 교정된 채점 방식에서 **현행 v9는 +0.0075, 다시 맞춘 값은 +0.0112로 +0.0037 차이**입니다(기준선 0.0032를 넘음). 다음 제출로 확인할 예정입니다.

같은 실험에서 형태 두 가지를 더 시험했습니다

기준을 무엇으로 삼나	맞춰진 배율	교차검증 개선	결과
예측값 크기 (현행 방식)	(0.60, 0.93, 1.18)	+0.0112	최선
날짜가 얼마나 먼가 (+1~2 / +3~5 / +6~7일)	(1.02, 0.97, 1.00)	-0.0001	기각
모델이 얼마나 확신하는가 (시드 5개 예측이 얼마나 흩어졌나)	(0.93, 1.02, 1.20)	+0.0046	현행보다 나쁨

날짜 기준은 세 배율이 거의 같게 나왔습니다(1.02 / 0.97 / 1.00) — **먼 날짜라고 해서 더 압축되어 있지는 않다**는 뜻입니다. 확신도 기준은 배율이 뚜렷이 갈렸지만(0.93 / 1.02 / 1.20) 예측값 크기 기준보다 못했습니다. 확신이 없는 칸과 예측값이 큰 칸이 상당히 겹치기 때문으로 보입니다.

4.6 더 정교하게 만들려는 시도는 전부 성적이 나아지지 않았습니다

시도	같은 데이터로 채점	교차검증	결과
구간을 5개로 늘림 (자유도 5)	-0.0009	-0.0060 (0/6)	기각
영업장 군집별로 따로 보정 (자유도 9)	+0.0009	-0.0038 (0/6)	기각
구간 없이 매끄럽게 늘리기 (자유도 1)	+0.0023	+0.0000	기각
구간 경계까지 최적화	6가지 분할 모두 원래 값(2, 10)으로 수렴		현행 유지

표의 "교차검증" 열은 폴드 2개에서 값을 정하고 나머지 폴드 2개에서 채점한 결과입니다. 괄호의 (0/6)은 여섯 가지 분할 방식 중 개선된 것이 0건이라는 뜻입니다. 자유도를 늘리면 같은 데이터로 채점한 값조차 거의 안 오르고 교차검증은 확실히 나빠집니다.

5. 시도했지만 성적이 개선되지 않은 세 가지

5.1 모델 설정 탐색 — 79가지, 순이익 0

XGBoost 무작위 탐색 40회, LightGBM 단계별 비교 9회, 작은 모델 비교 10회, 기타 20회를 돌렸습니다.

조절한 것	시험 범위	결과
트리 잎 개수	15 / 31 / 63 / 127 / 255 / 511	511은 확실히 나쁨. 15~255는 구분 불가
분기마다 쓰는 피쳐 비율	65% / 85%	큰 트리에선 효과 없음, 작은 트리에선 미미
잎 하나에 최소 몇 줄	5 ~ 160	아무 효과 없음 — 아래 설명
줄 샘플링 · 벌점 · 트리 개수 · 학습률	각각 5~6가지	효과 없음
알고리즘	LightGBM / XGBoost	구분 불가 (0.47008 vs 0.47119)

중간에 겪은 착오 — "잎 하나에 최소 몇 줄"이 아무 효과가 없었던 이유

XGBoost 탐색이 "잎 하나에 최소 40줄 → 10줄로 낮춰라"를 강하게 가리켜서 LightGBM에도 적용했는데, 10과 5가 소수점까지 완전히 같은 결과를 냈습니다. 버그를 의심했지만 단순한 산수였습니다.

학습 줄 185,000 ÷ 잎 255개 ≈ 잎 하나당 평균 725줄

잎을 255개에서 멈추는데 잎마다 700줄씩 들어가니 **최소 기준이 10이든 5든 한 번도 걸리지 않습니다.**

XGBoost는 트리를 키우는 방식이 달라서 같은 깊이 안에서 잎이 훨씬 잘게 쪼개지기 때문에 거기서는 의미가 있었던 것입니다. **다른 라이브러리의 결과는 방향의 참고이지 값을 그대로 옮길 수 있는 게 아닙니다.**

확실하게 알아낸 것 하나: 모델을 키우면 나빠진다

v4(잎 511개)가 **Public-Private 양쪽에서 가장 나빴습니다**(0.47644, 다음 순위와 0.0045 차이). 이걸 측정 오차가 아닙니다. 왜 그런지는 6장에서 설명합니다.

5.2 서로 다른 알고리즘의 예측을 평균하기

LightGBM · XGBoost · CatBoost 셋을 같은 조건(같은 피쳐 57개, 같은 학습 줄, 같은 시드)으로 학습해 비교했습니다.

	LightGBM	XGBoost	CatBoost
내부 검증 점수 (설정 조절 전)	0.4777	0.4739	0.4852
학습 시간 (같은 조건)	22초	68초	186초
세 모델이 서로 얼마나 다르게 틀리나	LGBM↔XGB 0.965 · LGBM↔Cat 0.944 · XGB↔Cat 0.932 (1에 가까울수록 똑같이 틀림)		

0.93~0.96은 "적당히 다르게 틀린다"는 수준이라 평균 낼 재료로 나쁘지 않습니다. 그런데 **세 예측을 그냥 평균했을 때 개선이 +0.0017로 기준선(0.0032) 미달이었습니다.** 각 모델에 다른 비중을 주는 것도 시도했지만 똑같이 평균하는 것보다 나을 게 없었고(최적 비중 +0.0016 ≤ 균등 +0.0017), 찾아진 비중이 폴드마다 뒤집혔습니다.

각자 최적으로 조절하면 모델들이 오히려 서로 닮아집니다

LGBM과 XGB가 얼마나 똑같이 틀리나

설정 조절 전

0.9648

각자 최적으로 조절한 후

0.9783 (더 닮아짐)

두 탐색이 각자 따로 했는데 같은 방향을 찾아냈습니다. "각자 최적으로 조절한 뒤 평균낸다"는 계획은 서로 닮음을 보장하지 않습니다 — 최적점이 하나뿐이면 조절할수록 모델들이 한 점으로 모입니다.

CatBoost는 설정 조절 없이 1회 측정만 하고 멈췄습니다. 제출에 들어간 적이 없습니다. 알고리즘 선택 자체가 구분 불가로 나왔고 설정 탐색에 7.6시간이 필요해 우선순위에서 밀렸습니다.

5.3 모델을 영업장별 · 날짜별로 쪼개기

영업장 성격이 워낙 다르니(1편 1.3) 따로 학습시키면 낫지 않을까 하는 발상이었습니다. 만들기 전에 여지가 있는지부터 했습니다. (전용 모델이 무엇인지는 2.3 참고)

대상	글로벌 모델	전용 모델	결과
계절 매장 (72품목)	0.5379	0.5292 (+0.0087)	폴드마다 방향이 달라 신뢰 불가
상시 영업 (98품목)	0.5200	0.5238 (-0.0038)	전용이 더 나쁨
연회장 (23품목)	0.3183	0.3226 (-0.0043)	전용이 더 나쁨

연회장은 목요일이 가장 높고 일요일이 가장 낮아 다른 8곳과 요일 패턴이 정반대입니다. 전용 모델이 이길 거라고 예상했는데 오히려 졌습니다. 트리는 이미 store 를 기준으로 맨 위에서 갈라지기 때문에 사실상 전용 모델 역할을 하고 있었고, 23품목만으로 따로 학습하면 다른 영업장 줄에서 배울 수 있는 일반 규칙(주말이 높다 등)까지 잃습니다.

날짜별로 쪼갠 것(+1일 ~ +7일용 모델 7개)은 7개 전부 글로벌 모델에게 졌습니다(-0.0004 ~ -0.0212). horizon이 이미 피쳐로 들어 있어 트리가 알아서 처리합니다.

쪼개기는 중단

쪼갠 모델들의 예측을 합쳐 전체 점수를 내도 가장 나은 안이 +0.0031(기준선 미달)이고, 그 개선마저 겨우 폴드 한 곳에서만 나왔습니다(나머지 3개 폴드는 전부 나빠짐). 26분 써서 며칠짜리 작업을 안 하게 됐습니다.

5.4 학습할 때 줄마다 다른 비중 주기

후보정이 "출력을 채점 방식에 맞춘 것"으로 크게 작용했으니, 학습 자체를 맞추면 어떨까 하는 발상이었습니다. 채점은 영업장 9곳이 각각 1/9을 가져가고, 영업장 안에서 품목들이 똑같이 나눠 갖습니다. 그런데 학습은 줄 개수대로 합니다.

영업장	품목 수	채점에서 품목 1개의 비중
담하	42	0.12 (기준)
포레스트릿	12	0.42 (3.5배)
화담숲주막	8	0.62 (5.2배)
화담숲카페	5	1.00 (8.4배)

화담숲카페 메뉴 하나가 담하 메뉴 하나보다 채점에서 8.4배 중요한데, 학습에서는 담하 줄이 훨씬 많아 반대로 배웁니다. 그래서 줄마다 $1 / (\text{영업장 품목수} \times \text{그 품목의 학습 줄 수})$ 를 비중으로 주고, 그 강도를 0(현행)부터 1(완전 정렬)까지 조절해 측정했습니다.

강도	후보정 없이	v9 후보정 적용
0.00 (현행)	0.5076	0.4993
0.25	0.5077	0.4989
0.50	0.5091	0.4995
0.75	0.5066 (+0.0010)	0.4974 (+0.0019)
1.00	0.5097 (-0.0022)	0.5010 (-0.0017)

기각했습니다. 가장 좋은 강도 0.75에서도 +0.0019로 기준선(0.0032) 미달이고, 폴드 4개의 방향이 엇갈립니다. 그리고 또 같은 양상 — 강도가 오를수록 겨울 폴드는 좋아지고(0.4494 → 0.4379) 봄 폴드는 나빠집니다(0.4919 → 0.4983).

왜 안 됐나 — 후보정과 무엇이 달랐나

트리는 이미 store 와 item_id 를 기준으로 갈라집니다. 비중으로 "이 품목을 더 보라"라고 하는 것은 모델이 이미 가진 정보를 다시 배분하는 것일 뿐 새로운 정보가 아닙니다. 오히려 줄이 적은 품목의 우연한 변동 을 키워 손해가 났습니다.

반면 후보정이 작용한 이유는 "모델이 스스로 알 수 없는 것"을 고쳤기 때문입니다 — 자기 출력에 일정한 치우침이 있다는 사실은 모델 스스로 알 수 없습니다. 1편의 원칙이 다시 확인됐습니다: 모델이 구조적으로 접근할 수 없는 정보만 도움이 됩니다.

6. 가장 중요한 발견 — 내부 검증이 실제 성적과 반대로 움직였다

6.1 무슨 일이 있었나

LightGBM 설정을 바꿔 내부 검증 점수를 크게 올렸습니다($0.4709 \rightarrow 0.4640$). 확인용 시드까지 따로 떼어 검증한 결과였습니다. 그런데 제출하니 **대회 점수는 오히려 나빠졌습니다**($0.4818 \rightarrow 0.4871$).

제출 6건이 모이자 양상이 드러났습니다.

제출	모델 크기	내부 검증 순위	실제 순위
v4	가장 큼 (약 511개)	1위	5위
v5	—	2위	4위
v7	중간	3위	3위
v3	중간	4위	1위
v1	가장 작음	5위(꼴찌)	2위

내부 검증 순위와 실제 순위가 거의 정확히 뒤집혔습니다.

쓸모없는 정도가 아니라 **반대로 판단했으면 더 나았을** 상태였습니다. 그리고 내부 검증과 실제 성적의 차이가 모델이 클수록 함께 커졌습니다: $0.0093 \rightarrow 0.0120 \rightarrow 0.0188 \rightarrow 0.0230$.

6.2 확인용 시드를 따로 뒀는데 왜 못 막았나

1편에서 배운 대로, 우리는 **탐색용 시드와 확인용 시드를 분리**했습니다. 설정 40개를 시드 (42, 7)로 탐색하고, 최종 후보는 한 번도 쓰지 않은 시드 (2024, 913, 31)로 재확인했습니다. 확인용 시드에서도 개선이 유지됐습니다. 그런데도 실제 성적은 나빠졌습니다.

맞춰버리는 대상이 두 종류였기 때문입니다.

무엇에 맞춰버렸나	막는 방법	우리가 했나
주사위 눈 (같은 검증 구간, 다른 시드)	확인용 시드 분리	○ 했음
검증 구간 자체 (F2 + F3라는 특정 기간)	구간을 바꿔야 함	× 못 했음

확인용 시드는 "같은 기간에서 주사위만" 바꿉니다. 검증 구간이 고정된 채로 설정을 49개나 봤으니, 그 특정 기간에 잘 맞는 설정이 뵈었습니다.

6.3 원인 — 내부 검증과 실제 대회가 재는 "거리"가 다르다

	학습이 끝나는 시점	맞혀야 하는 시점	사이 거리
F2 폴드	2023-11	2023-11 ~ 2024-02	0~3개월
F3 폴드	2024-02	2024-02 ~ 2024-06	0~3.5개월
실제 제출	2024-06	2024-06 ~ 2025-05	0~11개월

내부 검증은 항상 학습 직후를 맞히는데, 실제 제출은 최대 11개월 뒤를 맞힙니다. 28일 창은 매번 새로 주어지니 입력 자체는 최신이지만, "창 통계를 보고 다음 주 판매량을 맞히는 관계" 자체가 시간이 지나며 조금씩 어긋납니다.

6.4 어떻게 확인했나 — 거리만 바꾼 실험

검증 구간을 봄으로 고정하고, 학습 줄 수를 185,335개로 똑같이 맞추고, 학습 구간만 앞뒤로 밀어서 거리만 바꿨습니다.

두 가지를 미리 차단했습니다. ① 거리와 계절이 섞이지 않도록 검증 구간을 고정했습니다(그냥 "가까운 검증 / 먼 검증"으로 나누면 가까운 쪽은 겨울, 먼 쪽은 봄이 됩니다). ② 거리와 데이터 양이 섞이지 않도록 줄 수를 맞췄습니다(origin 개수를 맞춰도 정답이 0인 줄을 뺀 뒤에는 15% 벌어져서, 많은 쪽을 무작위로 줄였습니다).

모델 크기	학습 직후 채점	3개월 뒤 채점
앞 63개 (작음)	0.5523 꼴찌	0.5036 1등
앞 127개	0.5453 (3위)	0.5087 (2위)
앞 255개	0.5447 (2위)	0.5122 (3위)
앞 511개 (큼)	0.5365 1등	0.5175 꼴찌

순위가 4→1, 3→2, 2→3, 1→4로 완전히 뒤집힙니다.

6.5 큰 모델이 먼 미래에서 무너지는 이유

학습 줄이 32만 개일 때, 앞 개수는 그것을 몇 칸으로 나누는가입니다.

앞 개수	칸 하나당 줄 수	만들어지는 규칙
15칸	21,000줄	"주말이고 최근 4주 평균이 높으면 → 평균의 1.2배"
511칸	626줄	"담하_공깃밥이고, 목요일이고, 12월이고, 최근 평균 8~12이고, 지난주 같은 요일에 9개 팔렸으면 → 11.4"

아래쪽은 아주 구체적이고, 626줄의 2023~2024년 데이터로 만들어졌습니다. 우리 데이터에 맞는 예가 있습니다.

화담숲 개장일:	2023년 → 3월 31일
	2024년 → 3월 29일
	2025년 → ??? (테스트 기간)

구체적인 규칙을 만드는 모델은 "3월 31일부터 매출 발생"에 가까운 것을 기억합니다. 2024년엔 이를 어긋나고 2025년엔 더 어긋날 수 있습니다. 큰 규칙만 만드는 모델은 그런 걸 기억할 칸이 없어서 "28일 창에 매출이 있으면 계속 팔린다" 같은 것만 배우는데, 창은 예측할 때마다 새로 주어지니 1년이 지나도 유효합니다.

6.6 검증 방식을 어떻게 바꿨나

검증 방식	구조	현재 취급
A. 기존 방식 (F2 + F3)	학습 직후 0~3개월을 채점	주력에서 제외
B. 거리를 벌린 방식 (FAR-봄, FAR-겨울)	3개월 건너뛰고 채점	방향은 맞지만, 거리를 벌리려면 학습 데이터를 줄여야 하는 한계
C. 실제 대회 점수	Public + Private	주력. 구분 가능 최소 차이 0.0032

A와 B는 "확실히 나빠지는 것 걸러내기"에만 쓰고, 실제 우열은 제출로 판정하기로 했습니다. 데이터가 1.5년뿐이라 "거리"와 "학습 데이터 양"을 분리할 수 없는 것이 근본 한계입니다 — 거리를 벌리려면 학습 구간을 앞으로 당겨야 하고, 그만큼 데이터가 줄어듭니다.

"두 방식이 같은 답을 내면 믿어도 된다"도 성립하지 않았습니다

A와 B가 처음으로 같은 답을 내서 "v7이 v1보다 낫다"고 했습니다(각각 +0.0033, +0.0057). 실제로는 v7 0.47195 > v1 0.47126으로 오히려 근소하게 나빴습니다. 두 방식이 공통으로 갖는 약점(학습 기간 1.5년, 데이터 부족)이 있으면 나란히 같이 틀립니다.

7. 진행하며 막혔던 지점

Q. 손실함수에 로그를 씌운다는 게 정확히 어디에 씌운다는 건가?

손실함수 안이 아니라, 손실함수가 보는 "정답"을 바꾼 것입니다.

```
lgb.Dataset(X, label=np.log1p(y))    ← 여기. 정답에 로그
lgb.train(objective="regression_l1")  ← 손실함수는 그냥 절대값
np.expm1(model.predict(X))           ← 예측을 원래 단위로 되돌림
```

결과적으로 모델이 줄이려는 대상이 $|예측 - 정답|$ (절대 오차)에서 $|\ln((1+예측)/(1+정답))|$ (비율의 로그)로 바뀝니다.

왜 그게 맞나: SMAPE 벌점 $2|p-y|/(p+y)$ 에서 $t = (p-y)/(p+y)$ 로 두면 벌점은 정확히 $2|t|$ 입니다. 그런데 $(1+t)/(1-t) = p/y$ 이므로

```
ln(p/y) = ln((1+t)/(1-t)) = 2t + (2/3)t^3 + (2/5)t^5 + ...
→ |ln(p/y)| = 2|t| + (t^3 이후의 작은 항)
               ↳ 이게 SMAPE 벌점
```

SMAPE는 사실상 "몇 배 틀렸나"를 재는 지표입니다. 그러니 로그를 씌운 공간에서 절대 오차를 줄이는 것이 곧 SMAPE를 줄이는 일이 됩니다. 숫자로도 맞습니다 — 실측 100·예측 110이면 SMAPE 0.0952, 로그 차이 0.0953입니다.

SMAPE를 직접 줄이도록 학습시킬 수는 없습니다. LightGBM 같은 모델은 손실함수를 두 번 미분한 값을 요구하는데, SMAPE는 분모에 $p+y$ 가 있어 계산이 지저분하고 예측이 작아지면 값이 발산합니다. 다루기 쉬운 대체품을 쓴 것입니다.

Q. 같은 로그인데 피처에 씌우는 것과 정답에 씌우는 건 왜 다른가?

	피처(입력)에 로그	정답에 로그
효과	없음. 트리는 값의 크기 순서만 보므로, 순서를 안 바꾸는 변환은 아무 영향이 없음	손실함수의 의미가 바뀜
확인한 것	가격에 로그를 씌웠더니 원본과 소수점 4자리까지 동일	다른 손실함수로 바꾸면 나빠짐
결론	스케일 조정·로그 불필요. 값을 묶는 범주화만 효과	핵심 설계 결정

Q. F2 · F3 같은 폴드가 데이터를 나눠 모델을 여러 개 만든다는 뜻인가?

아닙니다. 연습 시험을 여러 번 보는 것이고, 거기서 만든 모델은 전부 버립니다. 실제 제출 모델은 전체 데이터로 한 번 학습한 모델 하나입니다.

```
2023-01                                2024-06
F1 | █████ 학습 209 █████ || █████ 시험(가을) █████ |
F2 | █████ 학습 300 █████ || █████ 시험(겨울) █████ |
F3 | █████ 학습 391 █████ || █████ 시험(봄) █████ |
🌀 | █████ 학습 497 (전체) █████ || → 2024-06-16~
      (숫자는 origin 개수)
```

왜 무작위로 나누지 않나: 무작위로 자르면 (a) 2024년 데이터로 학습해 2023년을 맞추는 조합이 생기고, 더 문제는 (b) **같은 날짜의 정답이 학습 쪽과 검증 쪽 양쪽에 들어갑니다.** origin 2024-03-08 · horizon 7과 origin 2024-03-10 · horizon 5는 둘 다 **2024-03-15가 정답**이기 때문입니다.

왜 F1을 빼고 F2·F3만 쓰나: F1은 학습 origin이 209개로 **실제 제출(497개)의 42%뿐**입니다. 실제로 제출할 모델보다 훨씬 약한 모델을 시험하는 셈이라 판단이 뒤집힙니다. `item_id` 피처를 빼는 실험에서 F1은 "빼는 게 낫다", F2·F3는 "넣는 게 낫다"로 갈렸고, 실제 제출은 데이터가 더 많으니 F3의 답이 맞습니다.

Q. 시드 5개 평균이 무슨 뜻인가?

LightGBM은 학습할 때 무작위를 씁니다 — 트리마다 학습 줄의 85%만 뽑고, 분기마다 피처의 65%만 후보로 씁니다. **시드**가 그 무작위를 정합니다.

같은 데이터·같은 설정인데 시드가 다르면 다른 트리가 나오고 예측도 달라집니다. 5개를 학습해 예측을 평균 내면 **무작위에서 오는 흔들림은 상쇄되고 실제 신호만 남습니다.** 측정된 효과가 +0.005로, 이 프로젝트에서 세 번째로 큰 개선입니다.

실험할 때는 시드 2개로 빠르게 보고 제출할 때만 5개를 씁니다. 학습 시간이 5배이기 때문입니다.

Q. "예측값 전체에 같은 숫자 곱하기"가 왜 필요했나?

13,510칸을 다 예측한 다음 숫자 하나를 정해 전체에 곱하는 것입니다. 단순히 보이지만 **모델이 내는 값(중앙값)과 SMAPE가 원하는 값(중앙값보다 아래)의 차이를 한 번에 메우려는 시도**였습니다.

결과는 실패했고(폴드마다 최적 숫자가 1.01 / 0.98 / 0.98 / 0.80으로 흩어짐), 그 실패가 4장의 발견으로 이어졌습니다 — 구간마다 필요한 방향이 반대라 숫자 하나로는 안 되는 것이었습니다.

Q. Public 점수가 좋으면 Private도 좋은 것 아닌가?

왜 어긋납니까. 우리 제출 9건에서 v3는 **Public 1위**인데 **Private 5위**로 4계단 떨어졌습니다.

절대 점수는 더 크게 차이납니다 — 같은 파일인데 Public 0.4818, Private 0.4584로 0.023 벌어집니다. 이것은 모델 문제가 아니라 **두 반쪽의 난이도 차이**이고, 9건 전체에 +0.021로 고르게 나타났습니다. **Public과 Private을 서로 비교하면 안 되고, 모델끼리 비교는 반드시 같은 쪽에서 해야 합니다.**

8. 실험할 때 지키기로 한 규칙

1편의 "0.003보다 작은 차이는 판단하지 않는다"에 더해 세 가지가 늘었습니다.

규칙	왜
① 개선이 폴드 한 곳에 만 몰리면 신뢰하지 않는다	v4(거울 -0.0185 / 봄 -0.0036)와 v6(거울 -0.0138 / 봄 +0.0001) 둘 다 실제 대회에서 나빠졌다. 모델 쪼개기(+0.0031)도 같은 양상이라 기각했다.
② 조절할 숫자가 늘어나 면 반드시 교차검증한다	"폴드 2개에서 값을 정하고 나머지 2개에서 채점". 세 구간 보정은 같은 데이터 채점 +0.0086 / 교차검증 +0.0059로 통과했고, 다섯 구간과 군집별은 교차검증 값이 음수로 나와 기각했다.
③ 본격적으로 만들기 전 에 전제부터 쟀다	알고리즘 분업 계획을 11분에 판정했고, 모델 쪼개기를 26분에 판정했다. 둘 다 며칠짜리 작업을 안 하게 됐다.

③이 가장 값어치 있었습니다

"세 명이 알고리즘 하나씩 맡아 조절한 뒤 예측을 평균낸다"는 원래 계획을, 조절을 시작하기 전에 11분짜리 측정으로 판정했습니다. 결과는 "예측 평균의 이득 +0.002 · 알고리즘 3종 조절은 혼자 하룻밤이면 끝"이었고, 그래서 분업 계획 자체를 다시 짰습니다. 그냥 시작했다면 세 명이 며칠 써서 +0.002를 얻었을 것입니다.

9. 남은 일감과 논의할 것

9.1 지금까지의 정리

영역	상태	비고
채점 방식 이용 · 후보정	최대 성과	정답 0인 줄 제거 0.352 · 하한 1.0 · 구간 보정 0.0172
내부 검증 체계	재설계 완료	주력을 대회 점수로 옮김. 근본 한계는 데이터가 1.5년뿐이라는 것
모델 설정 탐색	종료	79가지 시험, 순이익 0
알고리즘 예측 평균	종료	단, 상위 두 모델의 조합은 미시도 (v5는 1위와 최하위를 섞은 것이었음)
모델 쪼개기	종료	영업장별 · 날짜별 둘 다 기각
학습 비중 조절	종료	5.4
피쳐	미착수	아래 9.2

9.2 아직 시도하지 않은 것

후보	내용	대회 규칙
리조트 전체 정보	2.2에서 확인했듯 지금 모델은 다른 영업장 정보를 전혀 안 봅니다. "오늘 리조트 전체가 붐빈다"를 구조적으로 알 수 없습니다. 9곳 합계와 "자기 영업장 ÷ 전체" 같은 피쳐를 넣으면 새 정보가 됩니다. 1편의 원칙("모델이 접근할 수 없는 정보만 이득")에 부합	안전
메뉴 이름에서 정보 뽑기	메뉴 이름을 글자 단위로 쪼개 비슷한 이름끼리 가깝게 만듭니다. 한국어 메뉴명은 의미가 글자에 드러납니다 — 아메리카노(ICE) / 아메리카노(HOT) , 한상 삼겹구이 / 삼겹살 1인분 . 목적은 드물게 팔리는 품목이 비슷한 품목에서 정보를 빌려오게 하는 것(채점이 드문 품목에도 한 표를 주므로)	안전 제공된 컬럼명만 사용
냉/온 음료 비율로 기온 추정	날씨 데이터가 금지이므로, 창 안의 아이스/핫 음료 비율로 대신합니다. 위 메뉴명 처리가 선행 조건	안전
사전학습 언어 모델 사용	글자가 안 겹쳐도 의미가 비슷한 경우(삼겹살↔목살)까지 잡습니다	확인 필요 모델 가중치가 "외부 데이터"에 해당할 수 있음. 규칙 원문 확인 요망
상위 두 모델 예측 평균 + 보정 다시 맞추기	v8(1위)과 v3(XGBoost)를 섞고 보정을 새로 맞춥니다. 예측을 평균내면 값이 더 가운데로 몰리는데(= 압축 심화), 보정이 그것을 퍼주므로 둘을 함께 쓰면 이득이 남을 수 있습니다	안전

9.3 팀에 묻고 싶은 것

- 제출 배분.** 하루 5회 제한입니다. 지금은 내부 검증을 믿을 수 없어 후보를 2~3개로 좁혀 제출로 판정하는 방식인데, 여러 명이 각자 제출하면 금방 소진됩니다. 누가 무엇을 언제 올릴지 정할 필요가 있습니다.

2. **사전학습 모델 사용 가능 여부.** 규칙 원문에 관련 조항이 있는지 확인이 필요합니다.
3. **각자 독립적으로 접근하는 것에 대해.** 지금까지 알아낸 바로는 **모델을 바꾸는 방향은 거의 다 소진됐고** 남은 것은 채점 방식 이용과 피쳐입니다. 다만 이건 **제가 잡은 접근 안에서의 결론**이라, 완전히 다른 각도(다른 피쳐 설계, 다른 문제 정의)에서 오는 개선은 여전히 열려 있습니다. 오히려 지금은 **제 파이프라인을 물려받지 않고 각자 처음부터 해보는 쪽이** 팀 전체로는 나올 수 있습니다.

각자 시작하신다면 이 세 가지만은 알고 시작하시면 좋겠습니다

1. **학습에서 정답이 0인 줄을 빼세요.** 채점에서 0인 칸이 제외되기 때문입니다. 이것만으로 $0.895 \rightarrow 0.543$ 입니다.
 2. **채점은 품목마다 한 표씩입니다.** 유효 날짜가 9개뿐인 드문 품목이 28개인 주력 품목과 동등합니다. 줄 단위로 평평하게 평균내면 실제보다 좋게 나옵니다.
 3. **개선 폭이 0.0032를 못 넘으면 판단할 수 없습니다.** 그보다 작은 차이를 쫓으면 우연을 쫓는 것입니다.
- 이미 시험해서 효과가 없던 것들: 작년 같은 시기 데이터 · 최근 데이터에 비중 더 주기 · l2/huber 손실함수 · 큰 값 잘라내기 · 예측값 전체에 같은 숫자 곱하기 · 알고리즘별 비중 최적화 · 기하평균 · 모델 크기 키우기